Compiler Design Lab – Viva Questions

1. Lexical Analyzer using Lex

Q1. What is Lex?
Lex is a lexical analyzer generator tool used to generate scanners (lexical analyzers) from a set of regular expressions.

Q2. What is the structure of a Lex program?
It has three sections separated by `%%`:

- **Definitions** (C declarations, regex macros)
- **Rules** (pattern–action pairs)
- **User code** (main function, helper functions)

Q3. What is `yytext`?
`yytext` is a global character pointer that holds the current matched token's text.

Q4. What is `yylex()`?
It is the main function generated by Lex that scans the input and executes actions on matches.

Q5. What is `yywrap()`?
Called at end of input. Returning 1 indicates no more input; returning 0 means continue scanning another file.

Q6. Difference between Lex and Yacc?

- **Lex** → generates lexical analyzer (tokenizer).
- **Yacc** → generates parser (syntax analyzer).

Q7. What is `yyleng`?
Length of the current token matched (stored in `yytext`).

Q8. How do you handle whitespace in Lex?
Using a rule like `[ \t\n]+ ;` (empty action discards it).

---

2. Count Lines, Words, Characters

Q1. Which regex matches a word?
`[^ \t\n]+` – one or more non-whitespace characters.

Q2. Why is `chars += yyleng` used?
Because `yyleng` gives the length of the matched token, we accumulate it to count characters.

Q3. How are lines counted?
By matching the newline character `\n` and incrementing the line counter.

Q4. What does `ECHO` do?
Prints the matched text to the output (default action in Lex).

Q5. What is the default action in Lex?
If no rule matches, the input is echoed to `yyout` (usually stdout).

---

3. Convert abc to ABC

Q1. How does Lex match substrings?
Lex uses regular expressions; `abc` matches the literal substring anywhere in input.

Q2. What if we want to match only whole words?
Use word boundaries: `\babc\b` or patterns with spaces.

Q3. What is the longest match rule in Lex?
When multiple patterns match, Lex chooses the **longest match**; ties are broken by **first rule** listed.

Q4. What is the difference between `ECHO` and `printf`?
`ECHO` prints the matched text directly; `printf` allows formatted output.

---

4. Count Vowels and Consonants

Q1. Why is `.` used with empty action?
To consume and ignore all other characters (digits, symbols, spaces).

Q2. What happens if we don't handle other characters?
They get echoed by default (unwanted output).

Q3. How to ignore case sensitivity?
Use `[aeiouAEIOU]` or use `-i` flag with flex.

Q4. What is the difference between `[a-z]` and `[a-zA-Z]`?
`[a-z]` matches only lowercase; `[a-zA-Z]` matches both cases.

---

5. YACC: Arithmetic Expression

Q1. What is Yacc?
Yet Another Compiler-Compiler – a parser generator that produces LALR(1) parsers.

Q2. What is `%token`?
Declares tokens that are returned by the lexer.

Q3. What is `%left` and `%right`?
Precedence and associativity declarations:

- `%left '+' '-'` → left associative, lower precedence
- `%left '*' '/'` → left associative, higher precedence

Q4. What does `$$` and `$1`, `$2`, `$3` mean?

- `$$` → value of the left-hand side non-terminal
- `$1, $2, $3` → values of RHS symbols in order

Q5. What is `%prec UMINUS`?
Assigns the precedence of `UMINUS` to a rule (used for unary minus).

Q6. What is `yyparse()`?
The main parsing function generated by Yacc.

Q7. What is `yyerror()`?
Called when a syntax error is detected.

Q8. What is the difference between Lex and Yacc?
Lex → tokenizer (regular expressions). Yacc → parser (context-free grammar).

---

6. YACC: Valid Identifier

Q1. What is a valid identifier in C?
Starts with a letter or underscore, followed by letters, digits, or underscores.

Q2. How is this represented in grammar?

```
id : LETTER rest ;
rest : LETTER rest | DIGIT rest | ;
```

Q3. Why is recursion used in the grammar?
To allow any number of letters/digits after the first letter.

Q4. What is left recursion vs right recursion?

- Left: `A → A α` (efficient for LALR parsers)
- Right: `A → α A` (uses more stack)

Q5. What is the start symbol?
The first non-terminal defined in the grammar (here `id`).

---

7. NFA to DFA

Q1. What is the difference between NFA and DFA?

- **NFA**: multiple transitions for same input, may have ε-transitions.
- **DFA**: exactly one transition per input symbol, no ε-transitions.

Q2. What is ε-closure?
Set of all states reachable from a state using only ε-transitions (including itself).

Q3. What is the subset construction algorithm?
Each DFA state is a subset of NFA states. Start with ε-closure of NFA start state; for each symbol, compute move + ε-closure.

Q4. How many states can a DFA have?
At most `2^n` states for an NFA with `n` states.

Q5. What is the time complexity of NFA to DFA conversion?
O(2^n) in worst case.

Q6. What is the role of `findstate()` and `addstate()`?

- `findstate()`: checks if a subset already exists in the DFA.
- `addstate()`: adds a new subset as a DFA state if not already present.

---

8. First and Follow

Q1. What is FIRST?
FIRST(X) = set of terminals that can begin strings derived from X. Includes ε if X derives ε.

Q2. What is FOLLOW?
FOLLOW(A) = set of terminals that can appear immediately after A in some sentential form.

Q3. What is the use of FIRST and FOLLOW?
Used in constructing predictive (LL(1)) parsing tables.

Q4. Rules for FIRST?

- If X is terminal → FIRST(X) = {X}
- If X → ε → add ε
- If X → Y1Y2...Yk → add FIRST(Y1) - {ε}; if Y1 derives ε, add FIRST(Y2), etc.

Q5. Rules for FOLLOW?

- Add $ to FOLLOW(start symbol)
- If A → αBβ → add FIRST(β) - {ε} to FOLLOW(B)
- If A → αB or β derives ε → add FOLLOW(A) to FOLLOW(B)

Q6. What is a nullable non-terminal?
A non-terminal that can derive ε.

---

9. Constant Propagation

Q1. What is constant propagation?
Compiler optimization that replaces variables known to hold constant values with those constants.

Q2. What is the benefit?
Enables further optimizations like constant folding, dead code elimination.

Q3. What is constant folding?
Evaluating constant expressions at compile time (e.g., `2+3` → `5`).

Q4. Difference between constant propagation and folding?

- **Propagation**: replacing variable with its constant value.
- **Folding**: computing constant expressions at compile time.

Q5. What is a three-address code?
Intermediate representation where each instruction has at most one operator and three operands (e.g., `t1 = a + b`).

---

10. Intermediate Code Generation

Q1. What is intermediate code?
A representation between source code and target code (e.g., three-address code, quadruples, triples).

Q2. What are the types of intermediate representations?

- Three-address code

- Quadruples
- Triples
- Indirect triples
- Abstract syntax trees (AST)

Q3. What is a three-address code?
Each statement has the form `x = y op z` with at most one operator.

Q4. Why is postfix used for code generation?
Postfix (reverse Polish) is easy to evaluate with a stack and naturally handles operator precedence.

Q5. What is the role of the stack in postfix evaluation?
Operands are pushed; when an operator appears, operands are popped, operation performed, result pushed.

Q6. What is the advantage of three-address code?

- Easy to optimize
- Easy to translate to target code
- Machine-independent

---

11. Backend: TAC to 8086 Assembly

Q1. What is the backend of a compiler?
The part that translates intermediate code to target machine code (code generation + optimization).

Q2. What is TAC?
Three-Address Code – intermediate representation with at most one operator per instruction.

Q3. What are the 8086 registers used?

- **AX**: accumulator (for arithmetic)
- **BX**: base register (for multiplication/division)
- **CX**, **DX**: count and data registers

Q4. Why is `MUL BX` used instead of `MUL AX, BX`?
8086's `MUL` instruction implicitly uses AX and the operand; result goes to DX:AX.

Q5. What is the difference between `MOV` and `ADD`?

- `MOV` → copies data (overwrites destination)
- `ADD` → adds and stores result in destination

Q6. How is division handled in 8086?
`DIV BX` divides DX:AX by BX; quotient in AX, remainder in DX.

Q7. What is the role of the accumulator (AX)?
Primary register for arithmetic and I/O operations.

Q8. What are the addressing modes in 8086?

- Register
- Immediate
- Direct
- Register indirect
- Based/indexed

General Viva Questions

Q1. What are the phases of a compiler?

1. Lexical Analysis
2. Syntax Analysis
3. Semantic Analysis
4. Intermediate Code Generation
5. Code Optimization
6. Code Generation

Q2. What is the difference between a compiler and an interpreter?

- **Compiler**: translates entire program at once, produces object code.
- **Interpreter**: executes line by line, no object code produced.

Q3. What is a token?
A meaningful unit of source code (keyword, identifier, operator, etc.).

Q4. What is a lexeme?
The actual character sequence matching a token pattern.

Q5. What is the difference between syntax and semantic analysis?

- **Syntax**: checks grammatical structure (parse tree).
- **Semantic**: checks meaning (type checking, scope).

Q6. What is a parse tree?
A tree representation of the syntactic structure of source code according to grammar.

Q7. What is an ambiguous grammar?
A grammar where a string can have more than one parse tree.

Q8. What is left factoring?
A grammar transformation to remove common prefixes in productions (helps LL(1) parsing).

Q9. What is left recursion elimination?
Transforming `A → Aα | β` to `A → βA'`, `A' → αA' | ε`.

Q10. What is an LL(1) parser?
A top-down parser that scans input left-to-right, produces leftmost derivation, uses 1 lookahead token.

Q11. What is an LR parser?
A bottom-up parser that scans left-to-right, produces rightmost derivation in reverse.

Q12. What is the difference between SLR, LALR, and CLR?

- **SLR**: simplest, uses FOLLOW sets.
- **LALR**: merged CLR states, most practical.
- **CLR**: most powerful, largest tables.

Q13. What is a symbol table?
A data structure used by compilers to store information about identifiers (type, scope, memory location).

Q14. What is type checking?
Verifying that operations are applied to compatible data types.

Q15. What is peephole optimization?
Local optimization that examines a small window of instructions and replaces them with faster equivalents.

Quick Reference Table

Tool	Purpose	Input	Output
Lex	Lexical analyzer generator	Regex rules	C code for scanner
Yacc	Parser generator	CFG rules	C code for parser
Flex	Fast Lex	Regex rules	C code
Bison	GNU Yacc	CFG rules	C code
